

Dependency Injection with Java code

Introduction:

Dependency Injection (DI) is a design pattern where an object's dependencies are supplied externally, rather than the object creating them itself. This leads to loose coupling, testability, and flexibility.

Without DI, classes create their own dependencies (tight coupling). With DI, dependencies are provided from outside (often through a constructor, setter, or framework).

1. What DI Really Is

DI is not a framework feature, it's a design principle:

You don't create dependencies inside a class; you give the class what it needs from the outside.

The class doesn't care who builds the dependency — it just uses it.

This makes systems flexible, testable, and replaceable in production without rewrites.

2. Example (Tight Coupling -> DI)

Tightly Coupled Code

```
class OrderService {  
    private PaymentProcessor processor = new StripeProcessor();  
    void checkout(double amount) { processor.process(amount); }  
}
```

****This class cannot switch payment gateways without changing code.**

With DI

```
interface PaymentProcessor { void process(double amount); }  
class StripeProcessor implements PaymentProcessor {  
    public void process(double amount) { /* Stripe API call */ }  
}  
  
class OrderService {  
    private final PaymentProcessor processor;  
    OrderService(PaymentProcessor processor) { this.processor = processor; }  
    void checkout(double amount) { processor.process(amount); }  
}  
  
public class App {  
    public static void main(String[] args) {  
        OrderService service = new OrderService(new StripeProcessor());  
        service.checkout(200.0);  
    }  
}
```

****Result: Swap StripeProcessor with PayPalProcessor or MockProcessor without touching OrderService.**

3. Interfaces vs Abstract Classes in DI

Interface = "I only care about what you do, not how you do it."

- Use when you expect multiple, unrelated implementations.
- Great for service contracts: logging, messaging, payments.

Abstract Class = "I'll give you some behavior, but you must fill in the rest."

- Use when implementations share common code or state.
- Great for base API clients, shared utility workflows.

4. Real-Life Production Scenario [Payment System]

Goal: An e-commerce platform with multiple payment methods, each sharing fraud detection and logging logic.

Interface: `PaymentProcessor`

- Why: Defines the contract. All processors must have `process(double amount)`.
- Allows switching PayPal ↔ Stripe ↔ Crypto without code change.

Abstract Class: `BasePaymentProcessor` implements `PaymentProcessor`

- Why: Fraud detection, logging, and request validation logic are identical across processors.
- Forces subclasses to only implement the payment API-specific call.

Example:

```
abstract class BasePaymentProcessor implements PaymentProcessor {
    public void process(double amount) {
        if (!fraudCheck(amount)) throw new RuntimeException("Fraud detected!");
        log("Processing: " + amount);
        doPayment(amount);
    }
    abstract void doPayment(double amount);
    private boolean fraudCheck(double amt) { return amt < 5000; }
    private void log(String msg) { System.out.println(msg); }
}
```

****Stripe and PayPal only implement `doPayment()`, nothing else.**

5. DI in Production { Why It Matters }

- Testing: Inject mocks to simulate external systems. No API calls in tests.
- Upgrades: Swap old components for new ones without touching business logic.
- Multi-environment builds: Inject dev/test/prod variants automatically.
- Failover: In distributed systems, inject alternative services if the primary fails.
- Security: Replace sensitive dependencies in staging to avoid production data leaks.

6. Questions (Do check alternate options (ways) to achieve certain "Hows" below in JAVA)

1. How do you guarantee thread safety for singleton dependencies in a high-load system?
2. How do you detect and fix circular dependencies in a DI-heavy architecture?
3. When would you deliberately choose field injection over constructor injection?
4. How would you implement runtime dependency resolution without any DI framework?
5. How do default methods in interfaces (Java 8+) change DI design compared to abstract classes?
6. How would you support tenant-specific dependency injection in a SaaS platform?
7. What's the danger of stateful dependencies in a distributed setup?
8. How can you hot-swap a dependency without restarting a production JVM?
9. How do you configure multiple implementations of the same interface without breaking autowiring?
10. How does Java's module system (JPMS) break reflection-based DI, and how do you fix it?

7. Answers

1. Make singletons stateless or use synchronization/lock-free data structures; for mutable state, use thread-local or scoped beans.
2. Use framework logs, constructor tracing, or dependency graphs; fix by introducing an intermediate service or using lazy resolution.
3. Legacy migration or extreme constructor parameter bloat; otherwise avoid — hides dependencies.
4. Use factories + reflection to load classes from config at runtime.
5. Interfaces can now hold reusable code via defaults, but still cannot store state — abstract classes remain better for shared state.
6. Use a TenantContext + custom bean scopes to load per-tenant implementations.
7. Risk of inconsistent state; mitigate via external state stores (DB/cache) or stateless design.
8. Use OSGi, dynamic proxies, or Spring Cloud Config with @RefreshScope.
9. Use @Qualifier, @Primary, or inject Map<String, Interface> for named selection.
10. JPMS blocks deep reflection; open required packages via module-info.java using opens keyword.